MINI PROJECT

TermsLens — Consumer Policy Analysis

Submitted by :

KAMAL SUNDAR SK [26120137]

Under the guidance of :

Dr. Rajalakshmi

For The Subject

Full Stack Programming



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING SHIV NADAR UNIVERSITY KALAVAKAM

Abstract

TermsLens is a full-stack mini project that helps consumers read terms, privacy notices and related online policies through evidence-backed findings. Instead of presenting a single unexplained score, it separates concerning permissions, favorable protections and unresolved wording, and connects each supported finding to its exact source passage. The central question is practical: what does the policy permit, what protection does it promise, and where is that statement written?

The application combines a React and TypeScript interface, a FastAPI service, a MySQL persistence layer and a separate Python worker. Local rules handle explicitly supported wording across eight clause groups. Consented Hybrid review can submit a bounded selection of unresolved clauses to Groq. Scoring, coverage and publication remain controlled by application code; the model does not assign the overall grade.

This report documents the implemented architecture, security controls, rate limiting, failure handling and user workflow. Images come from the supplied six-page printed output. That observed run requested Hybrid but saved a Rules-only result: 14 of 20 substantive clauses were rated, with nine concerns and five favorable findings. It demonstrates a useful local baseline and visible degradation, while successful live AI acceptance and general-policy quality remain unverified.

Project	Details
Title	Mini Project — TermsLens
Student	KAMAL SUNDAR SK [26120137]
Faculty guide	Dr. Rajalakshmi
Course	Full Stack Programming
Institution	Shiv Nadar University Chennai
Report date	30 October 2026
Demonstrated result	70% clause coverage; 14 rule findings; six unresolved clauses

Keywords: consumer policies, explainable analysis, rule-based systems, bounded AI, FastAPI, MySQL, consent, evidence, recovery.

Contents

1. Introduction and objectives 4

2. Features and consumer workflow 5

3. System architecture 6

4. Data design and persistence 7

5. Analysis, evidence and scoring 8

6. Security, privacy and trust boundaries 9

7. Failure engineering and diagnosis 11

8. Output and consumer interpretation 13

9. Verification and observed evidence 18

10. Limitations and future improvements 19

11. Conclusion 20

Appendix A. Running the demonstration 21

References and source notes 22

Figures 1–2 are architecture/recovery diagrams. Figures 3–8 reproduce the supplied output. Subsections continue within each numbered chapter.

1. Introduction and objectives

1.1 Problem statement

Online services commonly ask users to accept long policies before creating an account, purchasing a subscription or uploading content. Important permissions can be distributed across sections on advertising, licensing, cancellation and account closure. A favorable sentence can also be misunderstood: a promise not to sell information does not necessarily prohibit sharing it with advertisers, and the ability to cancel a subscription does not itself guarantee a refund.

TermsLens addresses this reading problem by turning supported policy passages into navigable findings. It assists inspection of the text; it does not determine a company's hidden intentions, establish legal enforceability or replace professional advice. Unfamiliar language and missing context are displayed as unresolved instead of being silently classified as harmless. [1, 2]

1.2 Project objectives

- Provide an authenticated workflow for submitting one English policy and reopening the saved analysis.
- Identify supported consumer risks and protections across eight defined clause groups.
- Explain each finding using an exact quote, a source label and a severity rating.
- Use optional external AI only with consent, verified configuration and a reservable budget.
- Preserve immutable policy versions and reports so later comparisons have a traceable basis.
- Make uncertainty, unavailable AI, failures and incomplete coverage visible to the user.

1.3 Scope and academic contribution

The project is a local educational demo. Its contribution is the integration of interpretation with durable jobs, ownership checks, budget accounting, evidence validation and usable report navigation. The implemented system supports Terms, Privacy, Cookies and Refund policy scopes. The demonstration policy is authored engineering material, not a human-labeled benchmark. General-purpose legal interpretation, broad semantic coverage and production deployment are outside the evidence established by this submission.

2. Features and consumer workflow

The interface starts with authentication and a policy form. The user chooses the policy type, pastes text or enters a public URL, and selects a review mode. Hybrid is the default when available, but its consent checkbox remains unchecked. The API returns a job identifier; the UI follows actual processing stages and opens the saved report when publication finishes.

Clause group	Consumer question
Sharing / selling	Can information be sold or passed to another organization?
Retention / deletion	How long can information be retained, and can it be deleted?
Tracking / consent	What tracking occurs, and does it require consent?
Content licensing / AI training	What rights are granted over uploaded content?
Renewal / refunds / cancellation	Can charges continue, and can payments be recovered?
Unilateral changes	Can the service change the agreement without notice?
Arbitration / class waivers	How are disputes handled and collective claims restricted?
Termination / export	Can access end abruptly, and can data be taken away?

Report and lifecycle features

The report begins with score bounds, coverage, key concerns, favorable findings and unresolved counts. Users can filter clauses by category, rating or source, inspect exact evidence, switch stored scoring profiles and compare the saved result with its rules-only baseline. Longer lists are paginated rather than displayed as one uninterrupted block.

Private history supports reopening reports. URL discovery offers up to four candidates for explicit selection. Refresh and reanalysis preserve separate source versions and interpretation revisions. Compatible reports can be compared; deletion uses confirmation and a recoverable cleanup status. These features are implemented, with the remaining runtime acceptance work recorded separately from the browser and unit-test evidence. [2–5]

3. System architecture

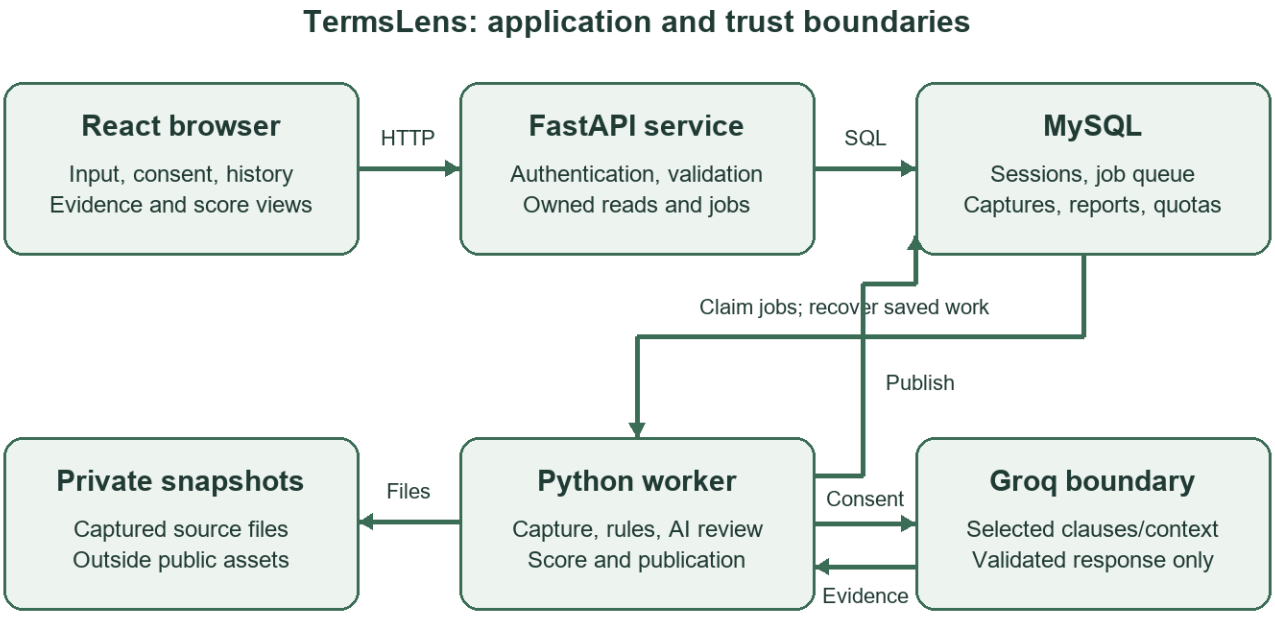


Figure 1. Architecture drawn from the implemented API, worker, database and provider boundaries; this is a technical diagram, not a runtime screenshot.

React 19 and TypeScript provide the browser interface, with Vite used for development and production assets. FastAPI validates requests and applies authentication, ownership, origin and CSRF checks. SQLAlchemy and Alembic provide relational persistence and schema migrations over MySQL 8. The development setup serves the built interface from Uvicorn on port 8000; Vite can run separately on port 5173. [2–4]

Analysis does not run inside the submission response. The API commits a job before returning HTTP 202. One normal Python worker claims queued work, captures the source, segments clauses, runs local rules, optionally coordinates AI, calculates the result and publishes the report. The browser polls the owned job rather than assuming progress from elapsed time.

The worker owns the external-processing boundary. Groq receives only selected clauses and required context. It has no database, filesystem or tool access. Raw fetched snapshots remain in private storage outside public assets. Saved reports are read through owner-checked API routes.

4. Data design and persistence

The relational design separates source material, interpretation and workflow state. A logical document may have several immutable captures, and each capture may have several analysis revisions. This distinction permits the system to report whether wording changed or the same wording was interpreted under different settings. [2, 3]

Record group	Responsibility
Users and sessions	Account identity, password hashes and hashed expiring session tokens.
Documents and versions	Ownership, source identity, ordered captures, cleaned text, hashes and capture metadata.
Jobs	Requested mode, consent, idempotency key, state, stage, attempts, lease and result link.
Analyses	Engine revisions, effective mode, scores, coverage, baseline and provenance.
Clauses and findings	Exact source offsets, substantive status, uncertainty and rule/AI evidence.
Category scores	Persisted category-level interpretation and score inputs.
AI calls and budget windows	Durable attempts, payload identity, responses, usage and reservations.
Comparisons and deletions	Saved comparisons, tombstones and recoverable cleanup receipts.
Fetch-host records	Shared host leases and request spacing for public URL operations.

Consistency boundaries

An owner-scoped idempotency key returns the existing job for an identical submission; reusing the key with different input is rejected. Publication commits the report, its job result link and terminal status atomically. A stale worker cannot publish after losing its lease. Comparisons require compatible source types, engine settings and effective modes.

Evidence offsets refer to the immutable cleaned capture using Unicode code points. The browser converts or slices these carefully so a non-BMP character, such as an emoji before a quote, does not shift the highlight. Profile switching uses stored findings and causes no additional provider call. Real MySQL restart, isolation and concurrency acceptance remain pending even though local persistence checks have passed.

5. Analysis, evidence and scoring

5.1 Rule-first interpretation

The current engine segments text into clauses, excludes recognized headings and checks a versioned set of supported statements. These initial engineering rules cover all eight groups, but their wording coverage is narrow. Actors, negation, qualifications and conflicting statements must not be discarded to manufacture a match. Supported findings store a category, severity, explanation and exact quote; unsupported clauses remain unrated.

5.2 Bounded Hybrid review

Unresolved clauses may be selected for Groq only after explicit consent and readiness checks. The payload includes IDs and context and treats the policy as untrusted data. The response must satisfy a strict schema, refer to selected IDs, use valid category/rating combinations and quote the supplied clause exactly. Uncertain, conflicting, duplicate or fabricated evidence is rejected. Source and model provenance distinguish accepted AI findings from local rules.

5.3 Deterministic score calculation

Higher scores are more favorable; higher clause severity indicates greater concern. Severity runs from 0 to 5: favorable 0, neutral 1, concerning 2–3 and invasive 4–5. Each applicable category uses supported severity maxima rather than averaging repeated favorable statements against a serious concern. Unknown evidence widens the possible score interval. [1, 4]

Measure	Interpretation
Coverage	Rated substantive clauses / all substantive clauses. This is not accuracy.
Upper score bound	100 minus 20 × the weighted mean of known category severity maxima.
Lower score bound	Uses worst-case severity for category uncertainty; fixed scope is retained.
Grade requirement	At least 80% coverage, consistent scope and both bounds in the same grade band.
Critical caps	Accepted data sale caps at 74; perpetual licensing plus mandatory arbitration caps at 59.

Terms uses all eight groups; Privacy uses groups 1, 2, 3, 4 and 6; Cookies uses 1, 2, 3 and 6; Refund uses 5 and 6. Balanced and Privacy-first views apply fixed weights to this scope. The model never assigns the overall grade. A report with no supported findings displays ‘Not enough evidence’ rather than treating an unknown upper bound as a favorable verdict.

6. Security, privacy and trust boundaries

6.1 Authentication and session protection

Passwords are hashed using Argon2id; the current hasher uses time cost 2, memory cost 19,456 KiB and parallelism 1. Random session tokens are generated using the platform secrets module, while only their SHA-256 hashes are stored in the database. Sessions expire after 12 hours by default. Logout removes the session record. Login failure messages do not identify which credential was incorrect. [4]

The session cookie is HttpOnly, SameSite=Strict and restricted to /api. State-changing authenticated requests require an allowed Origin and a matching CSRF token checked with a constant-time comparison. In the local HTTP demo, Secure cookies are disabled; an HTTPS deployment must enable them. This report does not claim that the local configuration is a production security audit.

6.2 Authorization and private data

Report, job, history, evidence, comparison and document operations derive the owner from the authenticated session. A document identifier supplied by a client is not sufficient authorization. Deletion tombstones block new attachments or refresh work while cleanup is pending. Test CLI settings select the isolated test database and clear Groq credentials, model, readiness path and enable flag.

6.3 Input, rendering and diagnostics

JSON models validate request types and lengths. A body-limit middleware bounds chunked requests before JSON parsing, including requests without a Content-Length header. React renders policy text as escaped content; fetched raw HTML is never rendered as a document page. Responses include nosniff, same-origin referrer policy and frame-denial headers; API responses use no-store caching.

Secrets remain in private .env or host environment storage. Git ignores private configuration, snapshots, local databases and generated caches. Diagnostics expose fixed codes and correlation identifiers, not passwords, API keys or policy bodies. The report and its build script use only supplied documents and recorded project evidence; they make no provider request.

6.4 Rate limiting and resource protection

Rate limits serve different purposes at different boundaries. The API limiter reduces repeated user/IP requests within one API process. The database ledger protects provider capacity across jobs and restarts. Neither mechanism is a substitute for the other. The values below describe current implementation and the recorded demo account, not universal Groq limits. [3, 4, 6]

Boundary	Current bound
Registration / login	5 registration requests/minute/IP; 10 login requests/minute/IP.
New submissions	10/minute/IP and 10/minute/user; at most five queued/running jobs per owner.
Discovery / comparison	Discovery: 3/minute/IP and user. Comparison: 10/minute/IP and user.
Request / policy input	1 MiB request body; policy ≤200,000 characters and ≤25,000 words; ≤2,000 clauses.
Public URL operation	8 requests including redirects/robots, 3 redirects, 2 MiB per decompressed response, 60 seconds.
AI per analysis	At most 2 requests and 12 selected clauses, at most 6 clauses per batch; 1 retry maximum.
AI token reservations	≤6,000 input/request; shared 10,000 input and 2,000 output across attempts. Output ≤1,000/request.
Recorded Groq account	openai/gpt-oss-20b: 30 RPM, 1,000 RPD, 8,000 TPM and 200,000 TPD.
API limiter storage	Bounded to 10,000 active buckets; cannot evict a live bucket to create fresh capacity.

Public URL and provider controls

URL fetching permits public HTTP(S) targets on ports 80/443, validates all DNS answers, pins the selected address while preserving Host/TLS checks, and revalidates redirects. Private, loopback, link-local and other forbidden addresses are rejected to reduce server-side request forgery risk. Robots rules, shared host leases and request spacing limit repeated fetching. Dynamic or blocked pages can be pasted manually.

The recent per-request increase from 5,000 to 6,000 does not raise the shared analysis total or account quotas. Token reservations use a conservative estimator and must cover context and output. Unknown or expired readiness permits zero calls. A larger first request can leave insufficient capacity for a retry; the ledger must refuse that retry. API request counters are in memory and reset with the API process, so a scaled deployment would need coordinated ingress limits.

7. Failure engineering and diagnosis

Failure engineering makes unsuccessful paths explicit and recoverable. TermsLens distinguishes configuration failure, unavailable external review, incomplete interpretation and an unusable job. A terminal request is not automatically proof that AI ran or that the policy received a reliable overall assessment. [2–5]

Observed or handled failure	Engineering response and lesson
Missing dependencies / MySQL login 1045	Safe diagnostics separate installation, login and schema checks. An administrator's successful login does not validate an application account.
Worker appears to hang	The worker remains open while waiting for queued jobs. Startup messaging explains that behavior; Ctrl+C stops it cleanly.
AI disabled in one terminal	PowerShell process environment overrides .env. Set the enable flag in both API and worker terminals before launching.
Readiness expired	Expose a fixed reason. Explicit renewal rechecks model/tokenizer, archives the old record and preserves quota history.
Context exceeds budget	Abstain without truncating an exception. Earlier 97- and 290-clause policies selected zero clauses for AI.
Provider timeout / unknown outcome	Retain reservations and record uncertainty. Do not automatically resend a possibly completed call.
Poor report readability	Move the overview first, group risks/protections/unknowns, paginate clauses and use exact-evidence dialogs.

Diagnose the boundary that failed

HTTP 200 from /api/health confirms the health response succeeded; it does not by itself mean AI is available. Inspect external_review.available and its fixed reason. Similarly, a successful model lookup verifies availability of the configured model, not an inference result. Request IDs connect API responses to jobs, while job IDs connect stage events, capture versions and published results.

The project preserves fallback reasons and requested/effective modes in saved reports. This makes a Rules-only result useful without mislabeling it as Hybrid success. General context selection remains a known blocker: the current rule guard can suppress the entire document and the AI payload can collect unrelated guarded paragraphs.

7.1 Recovery and child-process handling

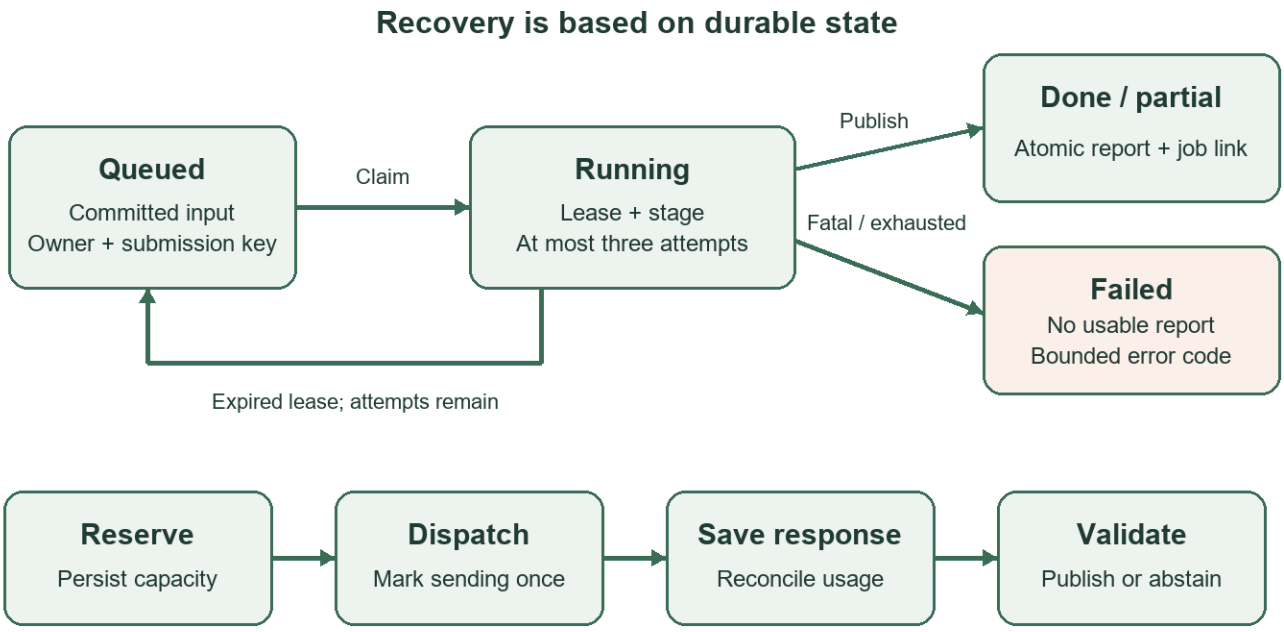


Figure 2. Job recovery and provider attempt ordering. Local tests cover these transitions; complete real-MySQL crash/race acceptance remains pending.

A worker claims a queued job with a lease and advances its processing stage. Recoverable interruptions may return work to the queue while attempts remain. The initial design permits one active worker and three job attempts. Losing or replacing a lease revokes publication rights so a stale process cannot create a second report.

Before calling Groq, the coordinator reserves capacity and persists the attempt. Dispatch is marked once; the response and usage are saved before interpretation. Compatible saved responses can be recovered without a fresh call. Unknown outcomes keep their charge, and completed or interrupted calls are not automatically replayed. A known 429 rejection may qualify for one bounded retry, subject to all remaining request/token quotas.

External work runs in bounded child processes. Parent cleanup kills and reaps children after timeout, communication failure or interruption; the Groq child also has an independent deadline watchdog. Credentials and payloads travel through standard input rather than command-line arguments. URL children use bounded request/result envelopes. Test workers clear provider settings so offline acceptance cannot consume production Groq capacity.

8. Output and consumer interpretation

The supplied sampe output.pdf is a six-page browser print captured on 30 October 2026 at approximately 09:20. Figures 3–8 are cropped from those pages for readability; their displayed results have not been changed. The captured report demonstrates the local baseline and a visible external-review fallback. [7]

Visible field	Observed value and meaning
Policy type	Privacy. The broad consumer example also contains terms-of-service categories.
Requested / saved mode	Hybrid / Rules-only. External review is shown as unavailable.
Evidence coverage	70%: 14 of 20 substantive clauses have supported findings.
Finding groups	Nine clauses need attention; five favorable findings; six unresolved clauses.
Displayed score	0–52 / 100, provisional, with no single letter grade.
Scope warning	Some findings fall outside the selected policy type; interpretation must retain that warning.

Why the policy type changes the range

The reproducible consumer-demo.md fixture is 434 words and should be submitted as Terms because it spans all eight groups. Its offline Terms baseline also rates 14 of 20 clauses, but its fixed category scope produces 0–42.5 rather than the printed Privacy range of 0–52. Both results are provisional. The screenshot remains valid evidence of what the application displayed; it is not relabeled to imply a different run.

What a consumer can learn

A no-sale promise appears alongside advertiser-sharing permission, so a user can distinguish sale from sharing. Cancellation rights and non-refundable payments are separate findings rather than a single reassuring claim. Perpetual content licensing and AI-training permissions remain prominent concerns, while deletion deadlines and consent-based cookies remain visible protections. The six nuanced clauses are review questions, not invented findings or guarantees about company intent.

The supplied print does not show accepted AI provenance. It therefore supports a working rule-based report and graceful fallback, not successful live Hybrid acceptance, measured accuracy or complete lifecycle verification.

8.1 Saved overview and uncertainty

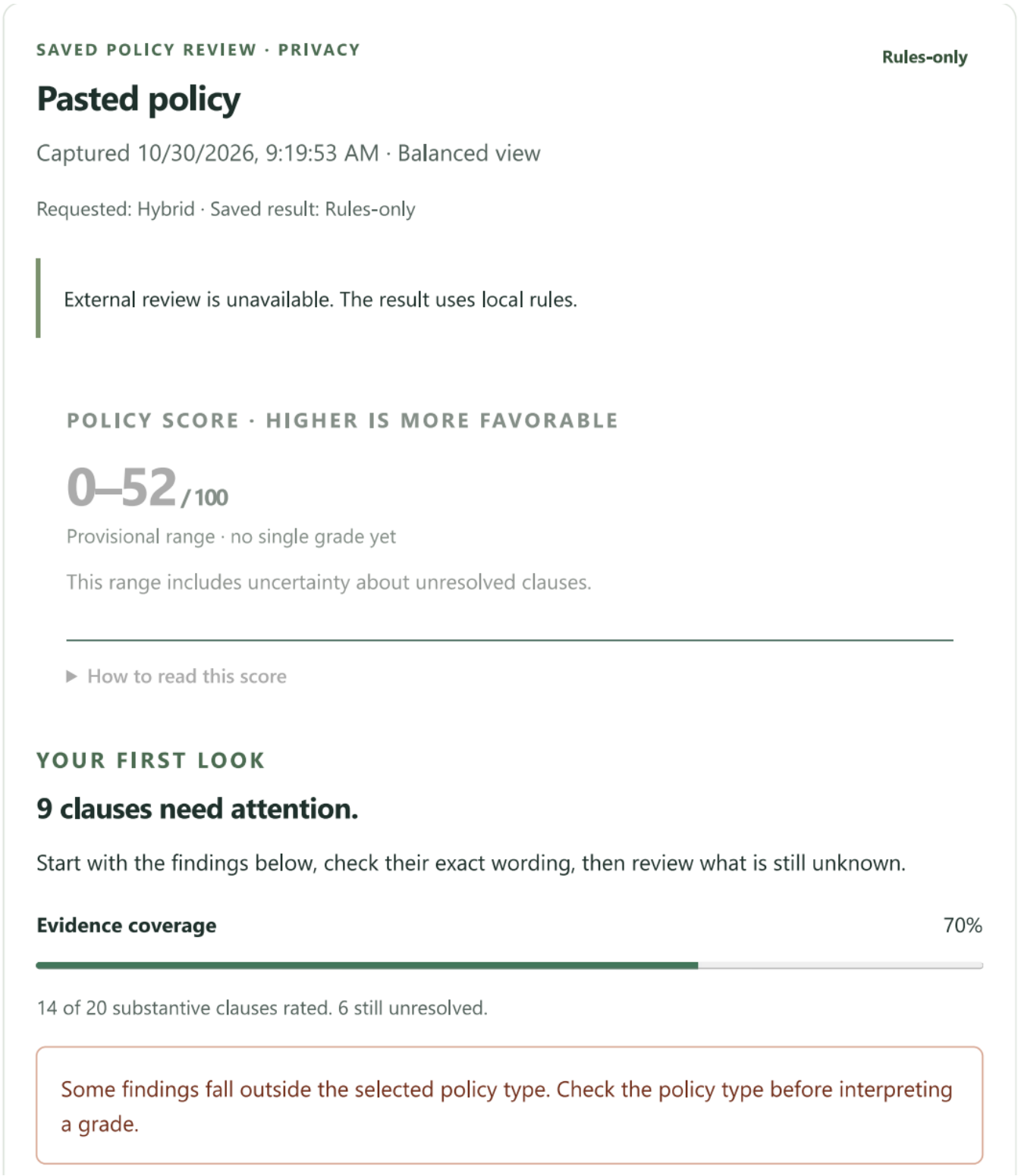


Figure 3. Supplied output, page 1: Hybrid was requested, Rules-only was saved, and 70% coverage is displayed. The Privacy scope warning and provisional 0–52 range are preserved.

8.2 Concerns and favorable findings

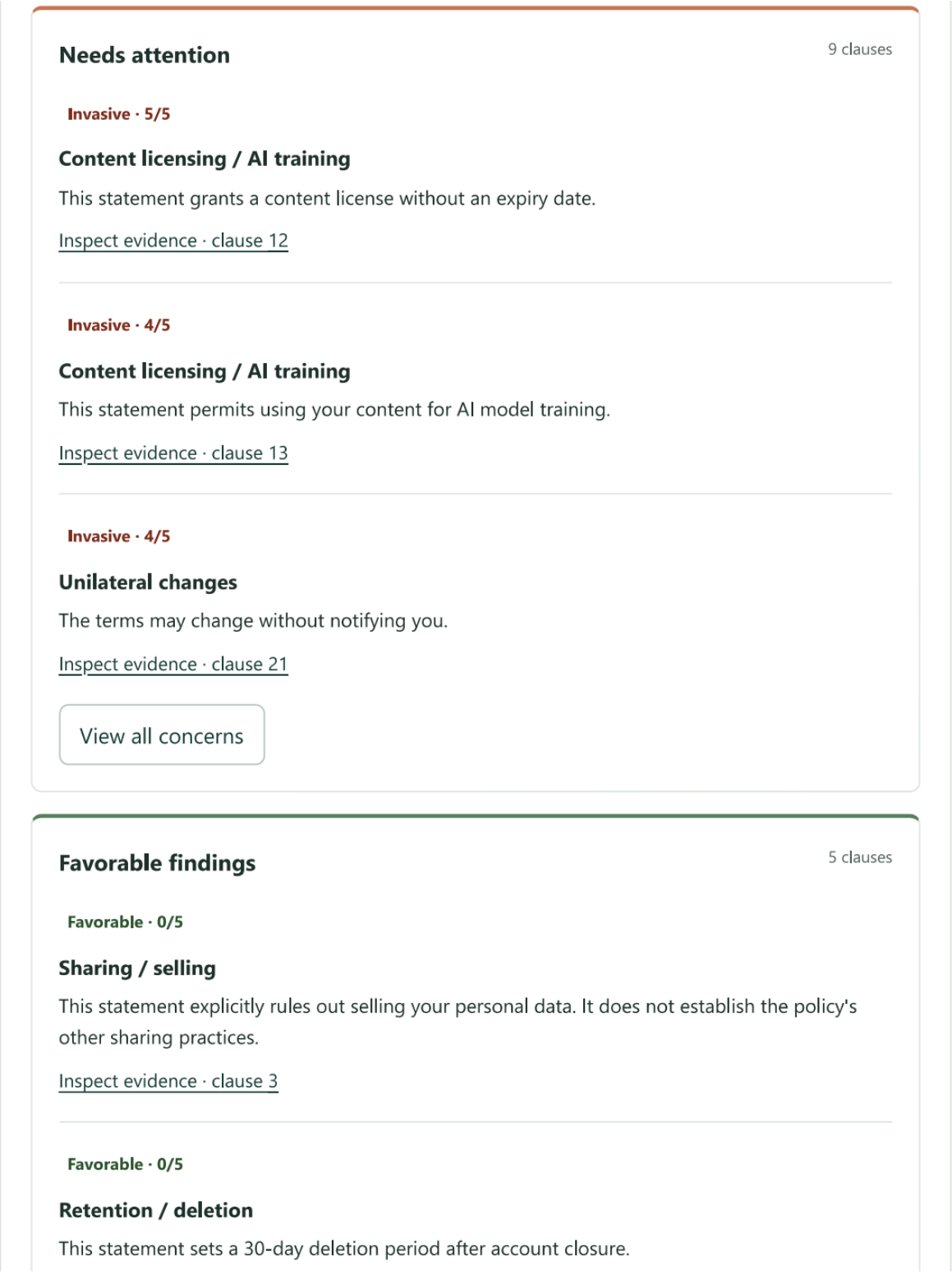


Figure 4. Supplied output, page 2: perpetual licensing, AI training and changes without notice lead the concern list. Favorable findings are presented separately and do not cancel these concerns.

8.3 Exact evidence and unresolved wording

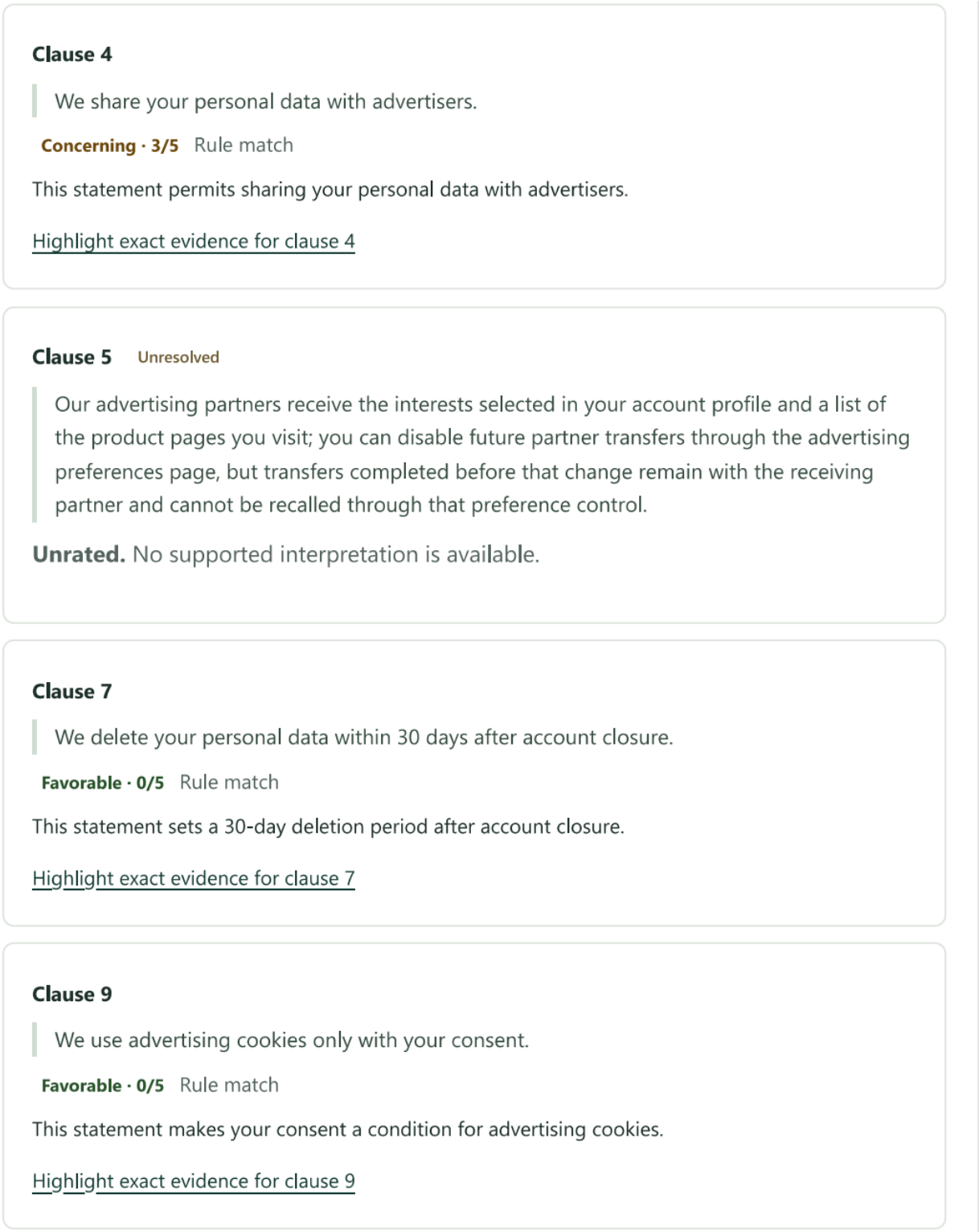


Figure 5. Supplied output, page 4: exact rule matches carry source labels and evidence links. The longer advertiser-transfer clause remains explicitly unresolved instead of receiving an unsupported rating.

8.4 Navigation, filters and saved actions

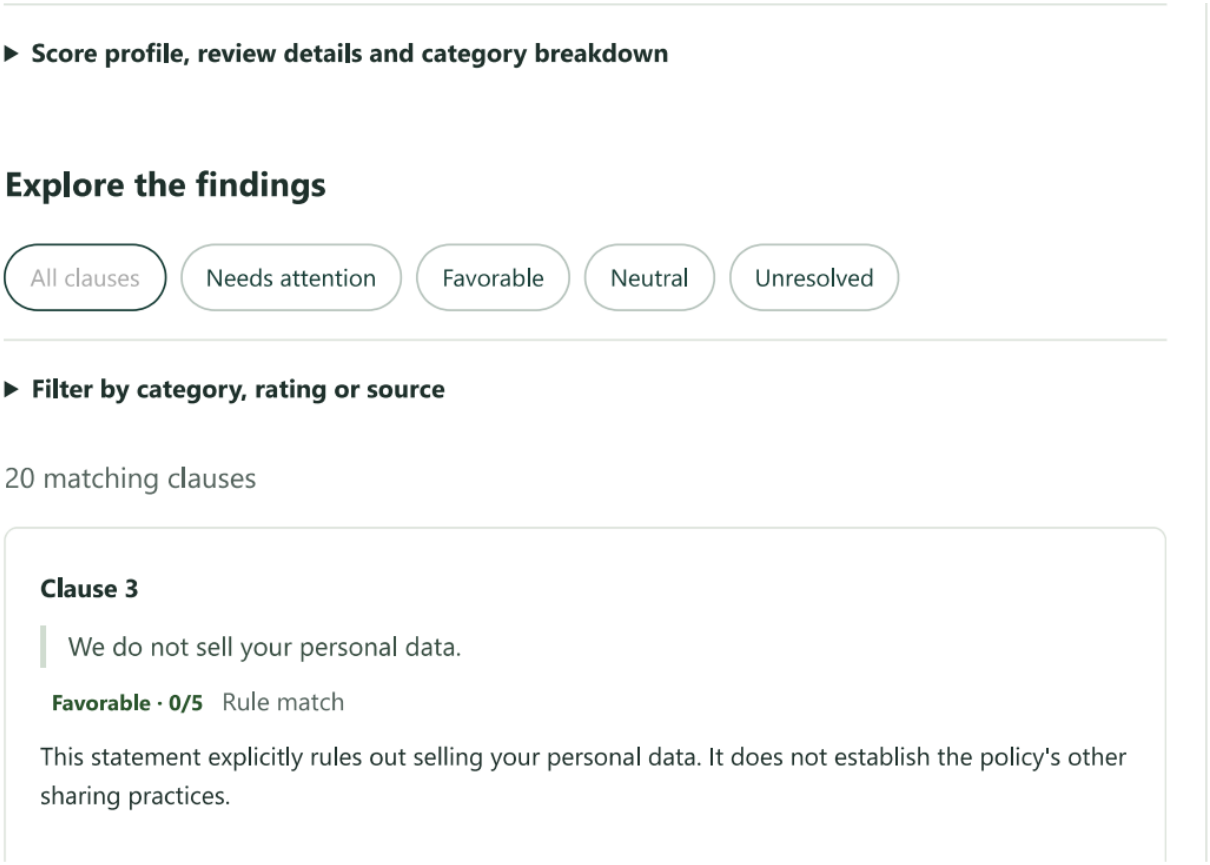


Figure 6. Supplied output, page 3: grouped filters and clause cards support focused inspection of the saved result.



Figure 7. Supplied output, page 5: paginated clauses and collapsed source/version/comparison/deletion controls keep secondary actions accessible.

The redesign places the consumer overview before technical detail. Evidence dialogs restore focus on close, severity is conveyed in text as well as color, and narrow-screen layout was checked with Playwright. A printed page cannot demonstrate keyboard behavior; that evidence comes from the recorded browser checks.

9. Verification and observed evidence

The following checks were recorded during implementation. They were not rerun merely to write this report. Test groups can overlap and should not be summed into a claim of independent acceptance coverage. Passing offline checks supports the tested contract, not every deployment or semantic interpretation. [5]

Evidence	Result and boundary
Contract / ledger / review / persistence / versions	71 tests passed after the allowance change, including shared totals, retries, stricter account limits and the consumer fixture.
Groq transport and child lifetime	29 tests passed with fake HTTP/local child processes; no live provider inference.
Frontend and browser	TypeScript/Vite build, 22 Node tests and Playwright smoke passed; API responses were mocked for browser checks.
Demo setup	Nine fake-provider tests passed for renewal, identity preservation, failure handling and secret-safe errors.
Database setup	User-observed MySQL 8.0.28 logins and both schema checks passed at revision 20261029_lifecycle.
Runtime print	Shows 14 rule findings, explicit AI fallback, scope warning and a deletion receipt; not broad acceptance.

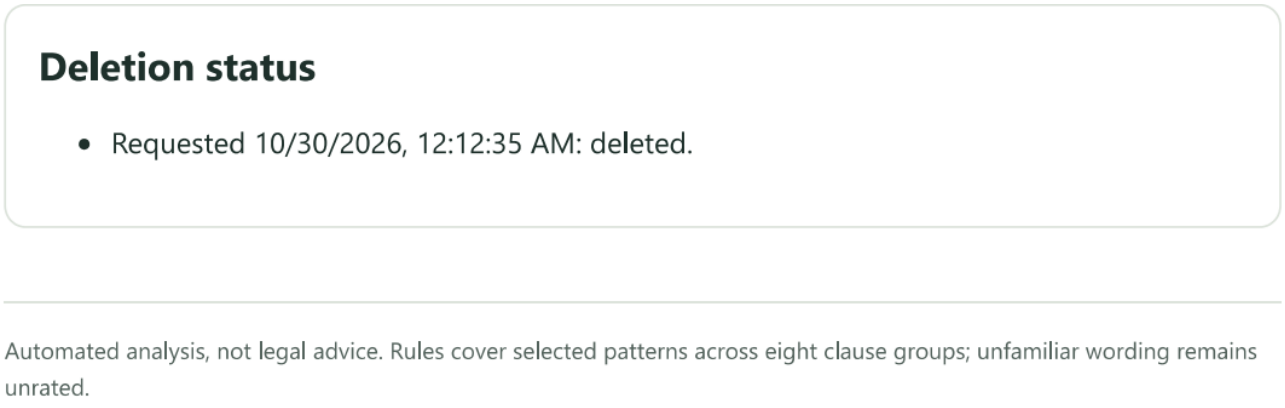


Figure 8. Supplied output, page 6: a deletion status receipt is visible. This screenshot alone does not establish cleanup correctness under a crash or simultaneous access.

Frozen plan/source hashes and document-size checks also passed. Outstanding work includes complete real-MySQL worker/restart/two-user/race verification, current live Hybrid evidence, human-labeled evaluation and the fixed performance benchmark.

10. Limitations and future improvements

10.1 Interpretation and coverage

The current rules recognize a limited set of supported engineering statements. The reviewed exact-match library is still empty, and no human-labeled held-out evaluation has established precision, recall, macro F1 or semantic coverage. The 70% demo coverage describes one deliberately constructed input; it must not be reported as 70% accuracy or expected performance on arbitrary policies.

Context handling is currently too broad. A definition or exception anywhere can guard all local rules, while the AI request collects guarded paragraphs across the document. The reported 97-clause policy still produces no AI batch under the modest allowance increase. Fixing this requires section-aware context and relevant definitions without dropping qualifications; raising a budget alone does not resolve the structural problem.

10.2 AI, deployment and operations

AI review is bounded by consent, expiring readiness, provider access, account usage and per-analysis limits. Twelve selected clauses cannot establish high whole-policy coverage for a long document without stronger local rules. Provider refusals, timeouts and validation rejection may preserve a Rules-only result. The supplied output does not establish successful live AI value.

The application uses a single API process and normal worker for the local demo. API rate counters are not distributed. Public HTTPS deployment would require suitable TLS/cookie configuration, operational monitoring, backup/restore arrangements and broader security testing. Dynamic or blocked websites may require pasted text. English is the supported interpretation scope; jurisdiction-specific enforceability is not determined.

10.3 Prioritized next work

- Complete real-stack isolation, restart, race and cleanup acceptance with reproducible records.
- Improve context selection and human-reviewed rules before promising broader policy coverage.
- Evaluate both modes using the frozen plan's labeled data, held-out families and adversarial pairs.
- Measure the prescribed 10,000-word benchmark using warm-up plus 20 runs; report median/p95 and separate network, queue and AI time.
- Retain a small current live Hybrid example with accepted source/model evidence, without unnecessary repeated provider calls.

11. Conclusion

TermsLens demonstrates how a full-stack application can make consumer-policy analysis inspectable. The browser organizes risks, protections and unknowns; the API enforces access and input boundaries; the worker performs durable analysis; and the database retains source versions, evidence and processing state. Optional AI is contained by consent, quotas, schema validation and exact-source checks.

The supplied output establishes a useful local baseline: fourteen supported findings and clear navigation over a twenty-clause consumer example. It also exposes the limits honestly through Rules-only fallback, unresolved wording and a scope warning. The distinction matters because a completed request, a high coverage percentage and a working provider configuration answer different questions.

As a mini project, the work demonstrates application architecture, relational persistence, security controls, failure recovery and user-centered presentation. It is not yet an independently validated general policy assessor. The next improvements should strengthen real-stack acceptance and interpretation quality while preserving exact evidence and visible uncertainty.

Practical demonstration sequence

- Start the API and one normal worker; keep both terminals open.
- Log in, paste consumer-demo.md and select Terms so all eight groups are in scope.
- Use Hybrid only when available and check the explicit Groq consent box.
- Inspect concerns, protections, unresolved clauses and exact quotes before interpreting score bounds.
- Check requested versus saved mode and any AI source/model labels; reopen the saved report from history.

The report is based on the supplied cover/output PDFs, the frozen project plan and maintained implementation evidence as of 30 October 2026. It does not mark pending acceptance gates complete.

Appendix A. Running the demonstration

Use two PowerShell terminals at the project root. The existing MySQL setup and migrations are prerequisites; fresh-clone setup is documented in README.md and RUNNING.md. Stop an older API/worker with Ctrl+C before restarting to load code or environment changes. Never paste passwords or keys into these commands.

Terminal 1 — API

```
$env:TERMSLENS_GROQ_ENABLED = 'true'
.\.venv\Scripts\python.exe -m uvicorn app.main:app `
--app-dir apps/api --host 127.0.0.1 --port 8000
```

Terminal 2 — worker

```
$env:TERMSLENS_GROQ_ENABLED = 'true'
Push-Location apps/api
..\..\.venv\Scripts\python.exe -m app.worker
```

Terminal 3 — health and sample

```
(Invoke-RestMethod 'http://127.0.0.1:8000/api/health').external_review
Get-Content -LiteralPath './eval/fixtures/consumer-demo.md' `
-Raw -Encoding UTF8 | Set-Clipboard
```

Open http://127.0.0.1:8000. Choose Terms, paste the clipboard content, select Hybrid and give consent. The enable variable must be set in each launch terminal because PowerShell environment changes are process-local. An enable flag alone does not replace readiness or available quota.

If AI is unavailable

Read external_review.reason. For disabled, restart the corresponding process after setting its flag. For expired readiness, use the documented explicit renewal only with current account evidence. Do not delete the quota ledger or edit confirmation timestamps to force availability. The worker's waiting message is normal; --test deliberately disables provider settings.

Full commands, expected findings and troubleshooting remain in README.md, DEMO.md and architecture-debugging.md. Saved reports retain the original result; restarting a process does not retroactively add AI findings.

References and source notes

Project sources were used as the primary evidence. Figure captions identify original supplied output pages; diagrams were authored from code and workflow contracts. No new provider call was made to prepare the report.

[1] plan-proposal.md. Frozen TermsLens requirements, scoring rubric, operating bounds and five delivery gates.

[2] implementation-workflow.md. User flow, F1–F8 persistence boundaries, job states, recovery and verification contracts.

[3] architecture-debugging.md; url-fetching.md; versions.md; comparisons-deletion.md. Maintained architecture, diagnostics and lifecycle documentation.

[4] Application source: apps/api/app/{main,security,config,analysis,scoring,ai_contract,ai_ledger,ai_review,groq_config,groq_transport,worker}.py; URL modules; apps/web/src report and workflow components.

[5] verification-checklist.md and transfercontext.md. Recorded checks and pending runtime, provider, evaluation and benchmark acceptance, as inspected on 30 October 2026.

[6] DEMO.md; README.md; RUNNING.md; eval/fixtures/consumer-demo.md. Demo operation, synthetic example and expected local baseline.

[7] sampe output.pdf. User-supplied six-page browser print dated 30 October 2026, approximately 09:20. Used for Figures 3–8 without changing displayed results.

[8] Assignment_10_Report.pdf, page 1. User-supplied cover reference. Student, faculty, course and institutional details retained; heading adapted to Mini Project. The original file remains unchanged.

Artifact reproducibility

scripts/build_project_report.py regenerates this PDF and its editable Word companion using the supplied PDFs and installed local document libraries. Intermediate images and a source-hash manifest are stored under private var/report-build. The report source contains no credentials. The frozen plan, original assignment report and supplied sample output are preserved.

```
python scripts/build_project_report.py
.\.venv\Scripts\python.exe scripts/check_context.py
```